LAB DevOps — Pipeline CI/CD com IA Generativa

Instituto J&F • Centro de Educação para os Negócios • Outubro/2026

DevOps • GitHub Actions
CI/CD Automation

ALUNO / AUTOR	PROJETO ESCOLHIDO
Noé Carrion (noecarrion)	Flop+ (FlopPlus) — Streaming Web App
REPOSITÓRIO NO GITHUB	**BRANCH CRIADA E ENVIADA**
https://github.com/noecarrion/FlopPlus	`feature/ci-cd-pipeline`
LINK DO PULL REQUEST	**AMBIENTE DE DEPLOY (CD)**
https://github.com/noecarrion/FlopPlus/pull/new/feature/ci-cd-pipeline	https://noecarrion.github.io/FlopPlus/

1. Visão Geral do Projeto e Justificativa da Estratégia de Deployment

O projeto **Flop+** é uma aplicação web front-end temática desenvolvida em **HTML5**, **CSS3** e recursos multimídia (ícones FontAwesome, imagens AVIF/PNG), simulando com teor humorístico uma plataforma de streaming de produções cinematográficas com baixa bilheteria.

Justificativa da Estratégia de Deployment (GitHub Pages via GitHub Actions)

Em estrita conformidade com as diretrizes do Lab, a estratégia de Continuous Deployment (CD) foi criteriosamente selecionada com base nas características da aplicação:

- **Adequação Arquitetural:** Por se tratar de uma aplicação client-side puramente estática, a utilização de containers Docker ou servidores com backend dedicado (Render, Railway) adicionaria complexidade desnecessária e consumo de recursos sem benefício prático.
- **Alta Disponibilidade e Performance:** O **GitHub Pages** distribui a aplicação por CDN global (Fastly) de alta resiliência e baixa latência, provendo certificado SSL/TLS nativo automaticamente.
- **Segurança via OpenID Connect (OIDC):** O deployment utiliza as actions oficiais do GitHub (`actions/deploy-pages@v4` e `actions/upload-pages-artifact@v3`), operando via permissões nativas de token do repositório (`pages: write` e `id-token: write`). Isso elimina a necessidade de expor ou gerenciar chaves secretas (API Tokens) de terceiros.
- **Garantia de Qualidade (Quality Gate):** O pipeline foi configurado com a diretiva `needs: continuous-integration` e condicional `if: github.ref == 'refs/heads/main' && github.event_name == 'push'`, assegurando que o ambiente de produção nunca receberá código com falhas de validação sintática ou links quebrados.

2. Código Completo da Pipeline (.github/workflows/ci-cd.yml)

A pipeline foi estruturada em dois estágios independentes e sequenciais, implementada no arquivo `.github/workflows/ci-cd.yml` :

```yaml
name: Pipeline CI/CD - FlopPlus

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main
  workflow_dispatch:

# Permissões necessárias para publicação segura no GitHub Pages
permissions:
  contents: read
  pages: write
  id-token: write

# Permite apenas uma implantação concorrente no Pages
concurrency:
  group: 'pages'
  cancel-in-progress: false

jobs:
  # ═══════════════════════════════════════════════════════
  # ETAPA 1: CONTINUOUS INTEGRATION (CI)
```

```yaml
  # Executada a cada push e pull_request direcionados a main
  # ============================================================
  continuous-integration:
    name: CI - Build, Validacao e Testes
    runs-on: ubuntu-latest

    steps:
      - name: 1. Checkout do Repositorio
        uses: actions/checkout@v4

      - name: 2. Configurar Ambiente Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'

      - name: 3. Instalar Dependencias
        run: npm ci

      - name: 4. Executar Testes de Integridade de Links e Assets
        run: npm test

      - name: 5. Executar Validacao Sintatica HTML (HTMLHint)
        run: npm run lint:ci

  # ============================================================
  # ETAPA 2: CONTINUOUS DEPLOYMENT (CD)
  # Executada apenas em push na main, apos conclusao da CI
  # ============================================================
  continuous-deployment:
    name: CD - Deploy no GitHub Pages
    needs: continuous-integration
    if: github.ref == 'refs/heads/main' && github.event_name == 'push'
    runs-on: ubuntu-latest

    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}

    steps:
      - name: 1. Checkout do Repositorio
        uses: actions/checkout@v4

      - name: 2. Configurar GitHub Pages
        uses: actions/configure-pages@v5

      - name: 3. Fazer Upload do Artefato do Site
        uses: actions/upload-pages-artifact@v3
        with:
          path: '.'

      - name: 4. Realizar Deploy no GitHub Pages
        id: deployment
        uses: actions/deploy-pages@v4
```

3. Registro do Processo de Desenvolvimento com IA Generativa

Durante todas as etapas do laboratório, utilizou-se a ferramenta de **IA Generativa (Antigravity AI)** como apoio no desenvolvimento:

Iteração 1 — Diagnóstico Inicial do Projeto e Arquitetura da Pipeline — **Fase de Concepção**

Prompt Utilizado: "*Analise esse Lab anexado, avalie a estrutura do projeto FlopPlus e elabore a arquitetura da pipeline de CI/CD via GitHub Actions com teste e deployment adequados.*"

Resultado Obtido: A IA mapeou o projeto como estático (HTML/CSS), recomendou o GitHub Pages para o CD e propôs a divisão do workflow em dois jobs: CI (validação) e CD (deploy).

Problema Encontrado: O projeto original não possuía nenhum mecanismo de teste, linting ou arquivo de gerenciamento de dependências (`package.json`), o que deixaria o estágio de CI vazio ou meramente ilustrativo.

Alteração Realizada: Decidiu-se inicializar o projeto com Node.js/npm, instalar o linter `htmlhint` e desenvolver um script automatizado de validação de integridade.

Iteração 2 — Diagnóstico de Portabilidade de Caminhos (Path Resolution) — **Fase de Correção de Código**

Prompt Utilizado: "*Como assegurar que a navegação e recursos estáticos do FlopPlus funcionem perfeitamente no ambiente do GitHub Pages e localmente?*"

Resultado Obtido: A IA alertou que repositórios no GitHub Pages de projetos pessoais são hospedados no subcaminho `https://<usuario>.github.io/<repositorio>/` . Links que utilizam barra inicial na raiz (ex: `href="/index.html"`) buscam o domínio raiz e geram erro 404.

Problema Encontrado: Foram identificados 5 links com barras absolutas nos arquivos `index.html` , `HTML/cadastro.html` , `HTML/inicio.html` e `HTML/login.html` .

Alteração Realizada: Todos os 5 links foram refatorados para caminhos relativos portáteis (`../index.html` , `aboutme.html` , `HTML/aboutme.html`).

Iteração 3 — Implementação dos Testes e Tratamento de Erro de Caminho no Windows — **Fase de Implementação de Testes**

Prompt Utilizado: "*Desenvolva o teste automatizado em Node.js para checar tags HTML essenciais, integridade de assets e bloquear links absolutos, e configure o package.json.*"

Resultado Obtido: Código de `tests/validate-integrity.js` implementado com sucesso e configuração dos scripts no `package.json` .

Problema Encontrado: No Windows local, o diretório continha o caractere especial `&` (`Instituto J&F`). Os scripts `.cmd` gerados pelo npm falhavam ao interpretar o `&` como operador do terminal.

Alteração Realizada: Criado script `lint:ci` executando diretamente via `node node_modules/htmlhint/bin/htmlhint` , contornando a limitação do Windows e garantindo compatibilidade total tanto local quanto no runner Linux (Ubuntu) do GitHub Actions.

Iteração 4 — Refinamento de Segurança, Concorrência e Gatilhos do Workflow — **Fase de Pipeline CI/CD**

Prompt Utilizado: "*Estruture o arquivo ci-cd.yml garantindo permissões OIDC mínimas, controle de concorrência e que o CD ocorra estritamente após a conclusão com sucesso do CI na branch main.*"

Resultado Obtido: Arquivo `.github/workflows/ci-cd.yml` gerado com conformidade total às diretrizes do laboratório e documentação do GitHub Actions.

Problema Encontrado: Nenhum. Validação estática do YAML aprovada.

Alteração Realizada: Adição de bloco `concurrency` e anotações nos passos para clareza didática.

Iteração 5 — Versionamento, Branching e Envio Remoto — **Fase de Entrega e Git**

Prompt Utilizado: "*Crie a branch de trabalho solicitada pelo Lab, configure o .gitignore para ignorar artefatos locais/IDEs e faça o push para o repositório remoto.*"

Resultado Obtido: Criação da branch `feature/ci-cd-pipeline` , inclusão do arquivo `.gitignore` , commit com mensagem semântica e push bem-sucedido para `origin/feature/ci-cd-pipeline` .

Problema Encontrado: A pasta `.idea/` do WebStorm estava presente como untracked.

Alteração Realizada: Criação do `.gitignore` com regras para IDEs, `node_modules` e saídas de build.

4. Respostas aos Questionamentos Finais do Lab

Item Avaliado	Resposta Detalhada
1. Quantos prompts foram necessários para chegar à versão final?	Foram necessárias **5 iterações estruturadas (prompts)** com a IA Generativa, abrangendo: (1) concepção e arquitetura; (2) identificação e correção de rotas absolutas; (3) automação de testes de integridade e solução de conflito de caminhos; (4) estruturação de segurança e dependências no YAML; e (5) versionamento e publicação remota.

Item Avaliado	Resposta Detalhada
2. Quais foram os principais problemas encontrados?	1. **Ausência de automação de testes prévia:** O projeto não tinha testes ou linters configurados. 2. **Caminhos absolutos em links internos:** 5 links utilizavam `/`, o que quebraria em subdiretórios no GitHub Pages. 3. **Caractere especial no caminho local do Windows:** O diretório continha `Instituto J&F`, travando executáveis `.cmd` do npm. 4. **Arquivos de IDE não versionáveis:** Presença da pasta `.idea/` sem `.gitignore` na raiz.
3. Quais alterações foram necessárias?	• Criação de `.github/workflows/ci-cd.yml` com os jobs de CI e CD integrados. • Criação do `package.json` e `package-lock.json` com scripts `test`, `lint` e dependência do `htmlhint`. • Criação do script `tests/validate-integrity.js` com validação de 44 pontos de checagem. • Correção de links absolutos em `index.html`, `cadastro.html`, `inicio.html` e `login.html`. • Criação do arquivo `.gitignore` para ignorar `node_modules/` e `.idea/`.
4. Quais alterações foram realizadas manualmente?	• Inspeção visual da árvore de diretórios e do design original das páginas para certificar que as classes CSS e layout não sofreram regressão visual. • Validação das credenciais do Git (Git Credential Manager) e autorização de push remoto no GitHub. • Definição do nome padronizado da branch de trabalho: `feature/ci-cd-pipeline`.
5. Como você verificou que a pipeline estava funcionando corretamente?	A verificação foi executada em múltiplas camadas de garantia: 1. **Execução local dos scripts da CI:** Rodagem de `npm test` (44 verificações de integridade de links/assets aprovadas) e `npm run lint:ci` (5 arquivos HTML escaneados sem erros). 2. **Validação sintática do Workflow:** Verificação estrutural do schema YAML do GitHub Actions. 3. **Confirmação de Push Remoto:** Sucesso no envio para a branch `feature/ci-cd-pipeline` com link de Pull Request gerado pelo GitHub. 4. **Disponibilização do Guia de Ativação no Pages:** Instruções claras para seleção do GitHub Actions no menu *Settings* > *Pages*.

5. Breve Análise do Processo de Desenvolvimento com IA

A utilização de IA Generativa no ciclo de desenvolvimento de software e DevOps demonstrou ser altamente produtiva. A IA não apenas acelerou a escrita do arquivo de configuração do GitHub Actions, como atuou preventivamente ao diagnosticar problemas que comumente passam despercebidos por desenvolvedores, como o comportamento de links absolutos em subdomínios do GitHub Pages.

Por outro lado, o laboratório evidenciou que a atuação humana crítica permanece indispensável: a IA sugeriu inicialmente a execução direta do linter via binário do npm, que falhou no sistema operacional local devido à presença do caractere especial `&` no caminho do diretório. A intervenção humana foi fundamental para entender a causa raiz do erro de ambiente, solicitar a adaptação do script para execução direta via Node.js e validar a integridade dos testes antes da submissão remota.

6. Evidências de Execução e Passo a Passo de Ativação

Evidência 1: Execução com Sucesso dos Testes Locais de CI (npm test & npm run lint:ci)

```
> flopplus@1.0.0 test
> node tests/validate-integrity.js

============================================================
  FLOP+ - TESTE DE INTEGRIDADE E VALIDACAO DE ASSETS
============================================================

[1/3] Verificando arquivos essenciais da aplicacao ...
  ✓ index.html encontrado.
  ✓ LICENSE encontrado.
  ✓ CSS\style.css encontrado.
  ✓ IMG\logo.png encontrado.
  ✓ IMG\background.avif encontrado.
  ✓ index.html encontrado.
  ✓ HTML\aboutme.html encontrado.
```

```
  ✓ HTML\cadastro.html encontrado.
  ✓ HTML\inicio.html encontrado.
  ✓ HTML\login.html encontrado.

[2/3] Verificando estrutura e doctype dos arquivos HTML...
  ✓ Estrutura basica de index.html validada.
  ✓ Estrutura basica de HTML\aboutme.html validada.
  ✓ Estrutura basica de HTML\cadastro.html validada.
  ✓ Estrutura basica de HTML\inicio.html validada.
  ✓ Estrutura basica de HTML\login.html validada.

[3/3] Validando links (href) e assets (src)...

================================================
TOTAL DE VERIFICACOES REALIZADAS: 44
SUCESSO: Todos os arquivos, estruturas e links estao integros!
================================================

> flopplus@1.0.0 lint:ci
> node node_modules/htmlhint/bin/htmlhint index.html HTML/*.html

Scanned 5 files, no errors found (56 ms).
```

Evidência 2: Criação da Branch e Push com Sucesso para o Repositório Remoto

```
$ git checkout -b feature/ci-cd-pipeline
Switched to a new branch 'feature/ci-cd-pipeline'

$ git add .gitignore package.json package-lock.json tests/ .github/ HTML/ index.html
$ git commit -m "feat: implementa pipeline de CI/CD com GitHub Actions e GitHub Pages"
[feature/ci-cd-pipeline b9f6f7a] feat: implementa pipeline de CI/CD com GitHub Actions e GitHub Pages
 9 files changed, 605 insertions(+), 5 deletions(-)

$ git push -u origin feature/ci-cd-pipeline
branch 'feature/ci-cd-pipeline' set up to track 'origin/feature/ci-cd-pipeline'.
To https://github.com/noecarrion/FlopPlus.git
 * [new branch]      feature/ci-cd-pipeline → feature/ci-cd-pipeline
```

Passo a Passo para Ativação e Demonstração no GitHub

> **Como visualizar o pipeline e a aplicação publicada:**
>
> 1. No repositório github.com/noecarrion/FlopPlus, acesse **Settings > Pages**.
> 2. Na seção **Build and deployment > Source**, selecione a opção **"GitHub Actions"**.
> 3. Crie o **Pull Request** da branch `feature/ci-cd-pipeline` para a branch `main` (ou faça o merge direto).
> 4. Na aba **Actions**, visualize a execução automática do job `continuous-integration`.
> 5. Ao efetuar o merge na `main`, o job `continuous-deployment` será acionado automaticamente e publicará o site na URL: https://noecarrion.github.io/FlopPlus/.